

開始使用 Google Antigravity

來源：<https://codelabs.developers.google.com/getting-started-google-antigravity?hl=zh-tw>

步驟數：11

本程式碼研究室會逐步引導您安裝 Google Antigravity (首重代理程式的開發平台)，並體驗相關功能。

目錄

- 1. [1. 簡介](#)
- 2. [2. 安裝](#)
- 3. [3. 服務專員管理員](#)
- 4. [4. Antigravity 瀏覽器](#)
- 5. [5. 構件](#)
- 6. [6. 編輯者](#)
- 7. [7. 提供意見](#)
- 8. [8. 規則和工作流程](#)
- 9. [9. 技能](#)
- 10. [10. 安全性和控管機制](#)
- 11. [11. 結論](#)

步驟 1 · 約 5 分鐘

1. 簡介

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

在本程式碼研究室中，您將瞭解 [Google Antigravity](#)。這個代理式開發平台可將 IDE 帶往首重代理的紀元。

與只能自動完成程式碼行的標準程式設計助理不同，Antigravity 提供「任務控制中心」，可管理自主代理，規劃、編寫程式碼，甚至瀏覽網路，協助您建構專案。

Antigravity 的設計以代理程式為優先，這項技術的前提是，AI 不只是編寫程式碼的工具，而是自主的行為者，能夠規劃、執行、驗證及疊代複雜的工程工作，且幾乎不需要人為介入。

課程內容

- 安裝及設定 Antigravity。
- 瞭解 Antigravity 的重要概念，例如 Agent Manager、Editor 和 Browser 等。
- 根據您自己的規則和工作流程自訂 Antigravity，並考量安全性。

軟硬體需求

目前 Antigravity 僅適用於個人 Gmail 帳戶的預先發布版。並提供免費配額，供您使用頂尖模型。

您必須在本機系統上安裝 Antigravity。這項產品適用於 Mac、Windows 和特定 Linux 發行版本。除了自己的機器，您還需要下列項目：

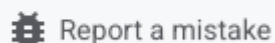
- Chrome 網路瀏覽器。
- Gmail 帳戶 (個人 Gmail 帳戶)。

本程式碼研究室適合各種程度的使用者和開發人員 (包括初學者) 參加。

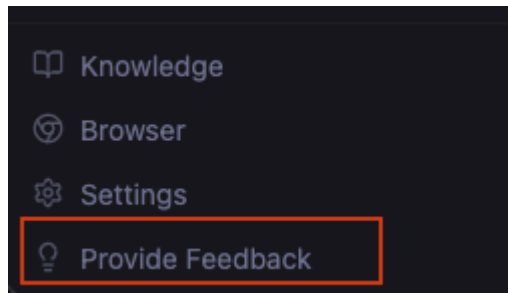
回報問題

在逐步完成程式碼研究室並使用 Antigravity 時，您可能會遇到問題。

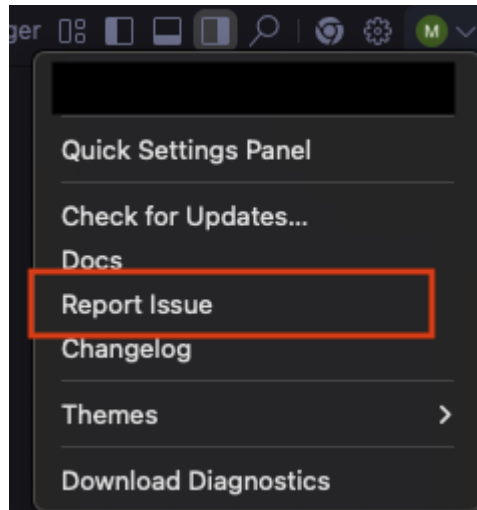
如要回報程式碼研究室相關問題 (錯字、錯誤的操作說明)，請點選本程式碼研究室左下角的 [Report a mistake](#) 按鈕開啟錯誤回報單：

A button with a bug icon and the text "Report a mistake".

如要回報 Antigravity 相關錯誤或要求功能，請在 Antigravity 內回報問題。您可以在 Agent Manager 中執行這項操作，方法是點選左下角的 [Provide Feedback](#) 連結：



你也可以點選個人資料圖示下方的 [Report Issue](#) 連結，前往編輯器：

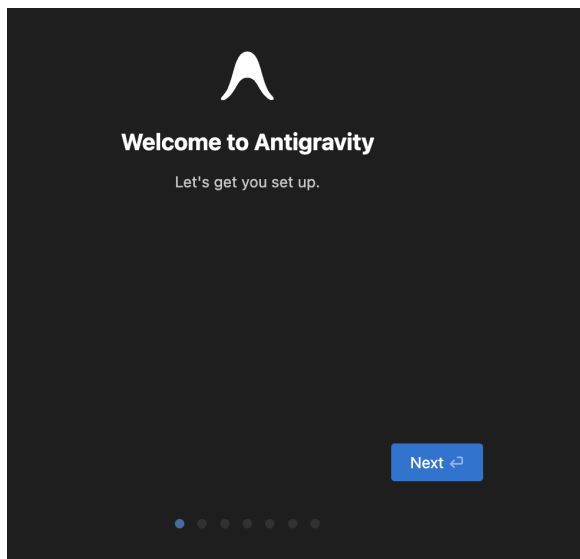


步驟 2 · 約 10 分鐘

2. 安裝

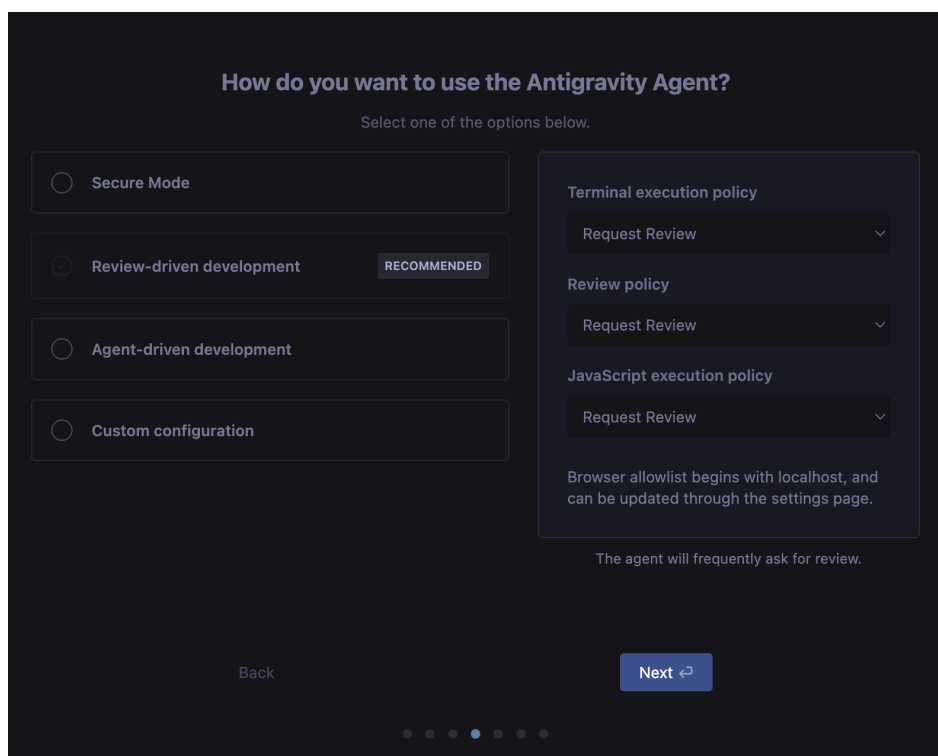
如果您尚未安裝 Antigravity，請先安裝。目前這項產品處於預覽階段，您可以使用個人 Gmail 帳戶開始使用。

前往[下載](#)頁面，然後點選適用於您情況的作業系統版本。啟動應用程式安裝程式，並在電腦上安裝。安裝完成後，請啟動 Antigravity 應用程式。畫面應會顯示如下所示的內容：



請每次都點選 **Next**。主要步驟如下：

- **選擇設定流程：**系統會顯示選項，讓您從現有的 VS Code 或 Cursor 設定匯入。我們將從頭開始。
- **選擇編輯器主題類型：**我們將選擇深色主題，但完全由您決定。
- **您想如何使用 Antigravity 代理程式？**



讓我們來深入瞭解。請注意，您隨時可以透過 Antigravity 使用者設定 (Linux/Windows：`Ctrl + ,`，Mac：`Cmd + ,`) 變更設定。

深入瞭解各個選項前，請先查看一些特定屬性 (對話方塊右側顯示的屬性)。

終端機執行政策

這是指授予 Agent 在終端機中執行指令 (應用程式/工具) 的權限：

- **一律繼續**：一律自動執行終端機指令 (可設定拒絕清單中的指令除外)。
- **要求審查**：執行終端機指令前，要求使用者審查並核准。

評論政策

Agent 執行工作時，會建立各種構件 (工作計畫、實作計畫等)。評論政策的設定方式可讓您決定由誰判斷是否需要審查。您是否一律要審查，或是讓代理程式決定。因此，這裡也有三種選項。

- **一律繼續**：代理程式絕不會要求審查。
- **代理決定**：代理會決定何時要求審查。
- **要求審查**：代理程式一律要求審查。

JavaScript 執行政策

啟用後，代理程式就能使用瀏覽器工具開啟網址、閱讀網頁，以及與瀏覽器內容互動。這項政策可控管瀏覽器執行 JavaScript 的方式。

- **一律繼續**：代理程式不會停止，要求在瀏覽器中執行 JavaScript 的權限。這可讓代理程式在瀏覽器中執行複雜動作和驗證作業時，享有最大自主權，但同時也最容易受到安全漏洞影響。
- **要求審查**：代理程式一律會停止運作，並要求授權才能在瀏覽器中執行 JavaScript 程式碼。
- **已停用**：代理程式絕不會在瀏覽器中執行 JavaScript 程式碼。

瞭解不同政策後，左側的 4 個選項就只是終端機執行、檢閱和 JavaScript 執行政策的特定設定，其中 3 個選項是預設設定，第 4 個選項則可完全自訂控制。我們提供這 4 個選項，讓您選擇要授予 Agent 多少自主權，以便在終端機中執行指令及取得審查過的構件，然後繼續執行工作。

這 4 個選項分別是：

- **安全模式**：安全模式可為 Agent 提供更完善的安全控管機制，方便您限制 Agent 存取外部資源和執行敏感作業。啟用安全模式後，系統會強制執行多項安全措施，保護您的環境。
- **以審查為導向的開發 (建議)**：代理會經常要求審查。
- **代理程式主導的開發**：代理程式絕不會要求審查。
- **自訂設定**

「以審查為導向的開發」選項是個不錯的平衡點，也是建議選項，因為代理程式可以做出決策，並回報給使用者以取得核准。

接著會進入「設定編輯器」設定頁面，您可以在這裡選擇下列偏好設定：

- **鍵盤快速鍵**：您可以設定鍵盤快速鍵。
- **擴充功能**：你可以安裝熱門語言和其他建議的擴充功能。
- **指令列**：您可以安裝指令列工具，透過 `agy` 開啟 Antigravity。

現在，您可以**登入 Google**。如先前所述，如果您有個人 Gmail 帳戶，即可免費使用 Antigravity 的預覽模式。**立即使用帳戶登入**。瀏覽器隨即開啟，供你登入。驗證成功後，您會看到類似下方的訊息，並返回 Antigravity 應用程式。隨波逐流。

最後是**使用條款**。你可以決定是否要啟用這項功能，然後按一下 `Next`。

這會帶您來到關鍵時刻，Antigravity 會等候與您合作。

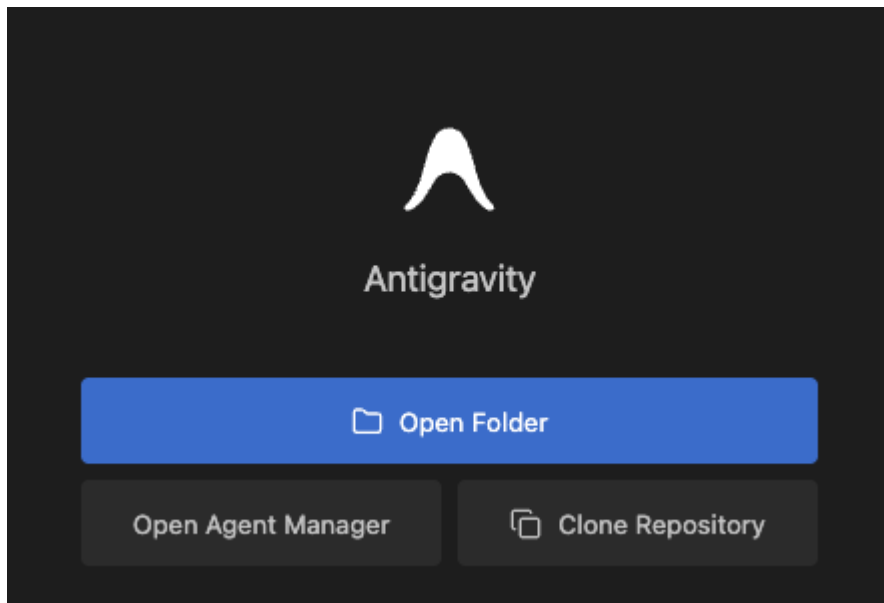
3. 服務專員管理員

我們準備好開始了！

Antigravity 是以開放原始碼的 Visual Studio Code (VS Code) 為基礎，但大幅改變使用者體驗，將重點從文字編輯轉移至代理程式管理。介面會分成兩個不同的主要視窗：「Editor」和「Agent Manager」。這種關注點的分離，反映了個人貢獻和工程管理之間的區別。

代理程式管理員：任務控管

啟動 Antigravity 後，使用者通常會看到「Open Folder」（開啟資料夾）、「Open Agent Manager」（開啟代理程式管理員）和「Clone Repository」（複製存放區）。



這個代理程式管理員就像任務控管資訊主頁，這項工具專為高階調度而設計，可讓開發人員生成、監控及與多個代理互動，這些代理會在不同工作區或工作非同步運作。

在這個檢視畫面中，開發人員扮演架構師的角色。他們會定義高層級目標，例如：

- 重構驗證模組
- 更新依附元件樹狀結構
- 為結帳 API 產生測試套件

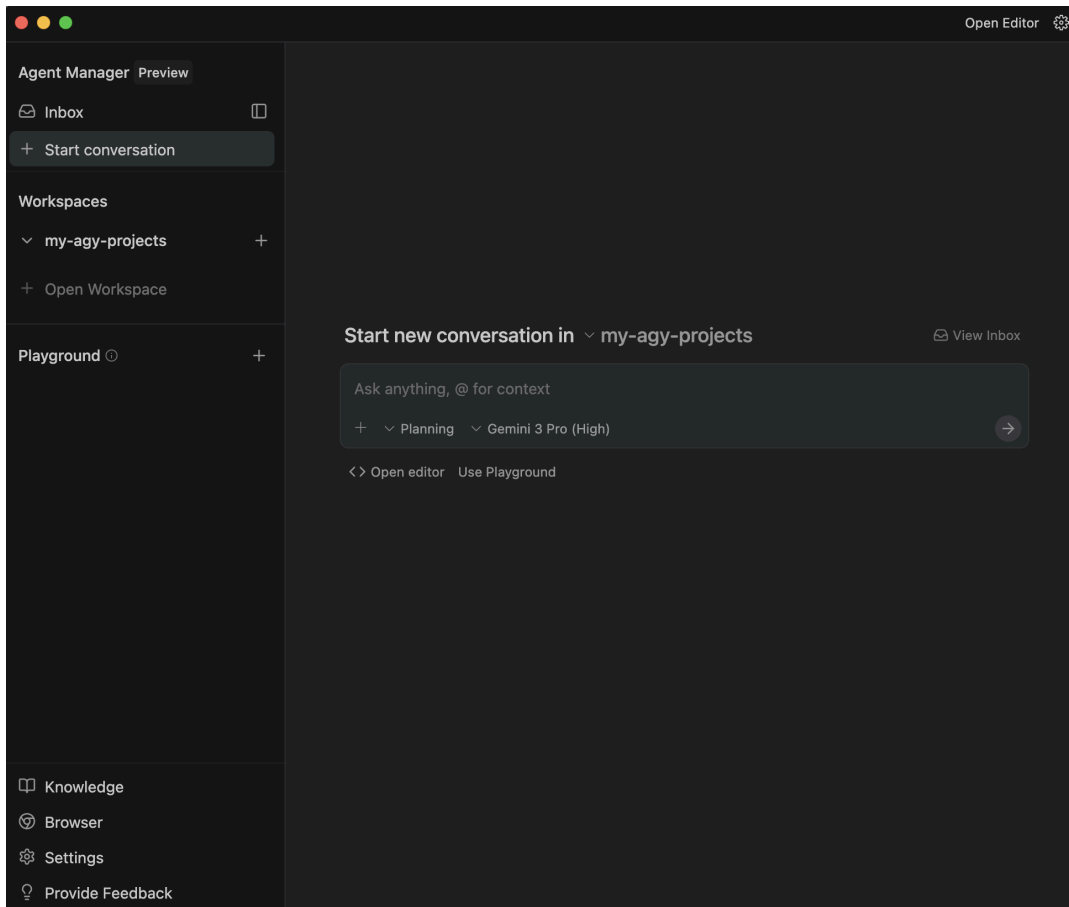
每項要求都會產生專屬的代理程式執行個體。使用者介面會以視覺化方式呈現這些平行工作流程，顯示每個代理的狀態、產生的構件（計畫、結果、差異）和任何待核准的要求。

先前的 IDE 較偏向聊天機器人體驗，屬於線性且同步，而這項架構解決了這項主要限制。在傳統的聊天介面中，開發人員必須等待 AI 生成程式碼，才能提出下一個問題。在 Antigravity 的管理員檢視畫面中，開發人員可以同時派遣五個不同的代理處理五個不同的錯誤，有效提高處理量。

按一下 `Open Folder` 即可開啟工作區。

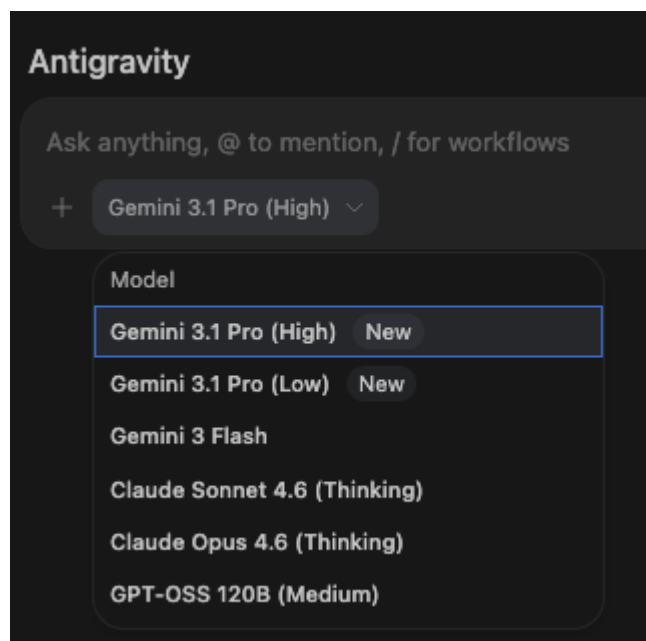
將工作區視為您在 VS Code 中所知的工作區，即可完成設定。因此，我們可以點按按鈕開啟本機資料夾，然後選取要開始使用的資料夾。以我為例，我在主資料夾中有名為 `my-agy-projects` 的資料夾，並選取該資料夾。你可以使用完全不同的資料夾。請注意，您也可以完全略過這個步驟，日後再開啟工作區。

完成這個步驟後，您會進入「Agent Manager」視窗，如下所示：

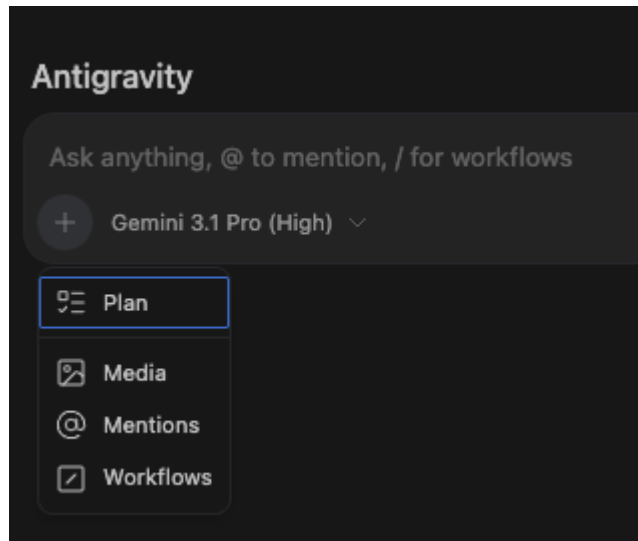


你會發現應用程式已準備就緒，可立即在所選工作區資料夾 (`my-agy-projects`) 中開始新的對話。您可以運用使用其他 AI 應用程式 (Cursor、Gemini CLI) 的現有知識，並使用 `@` 和其他方式，在提示中加入額外背景資訊。

請查看 `Model Selection` 下拉式選單。您可以透過「模型選取」下拉式選單，為 Agent 選擇目前可用的模型。清單如下所示：



同樣地，我們發現 Agent 會處於預設的 `Fast` 模式。但我們也可以選擇 `Plan` 模式。

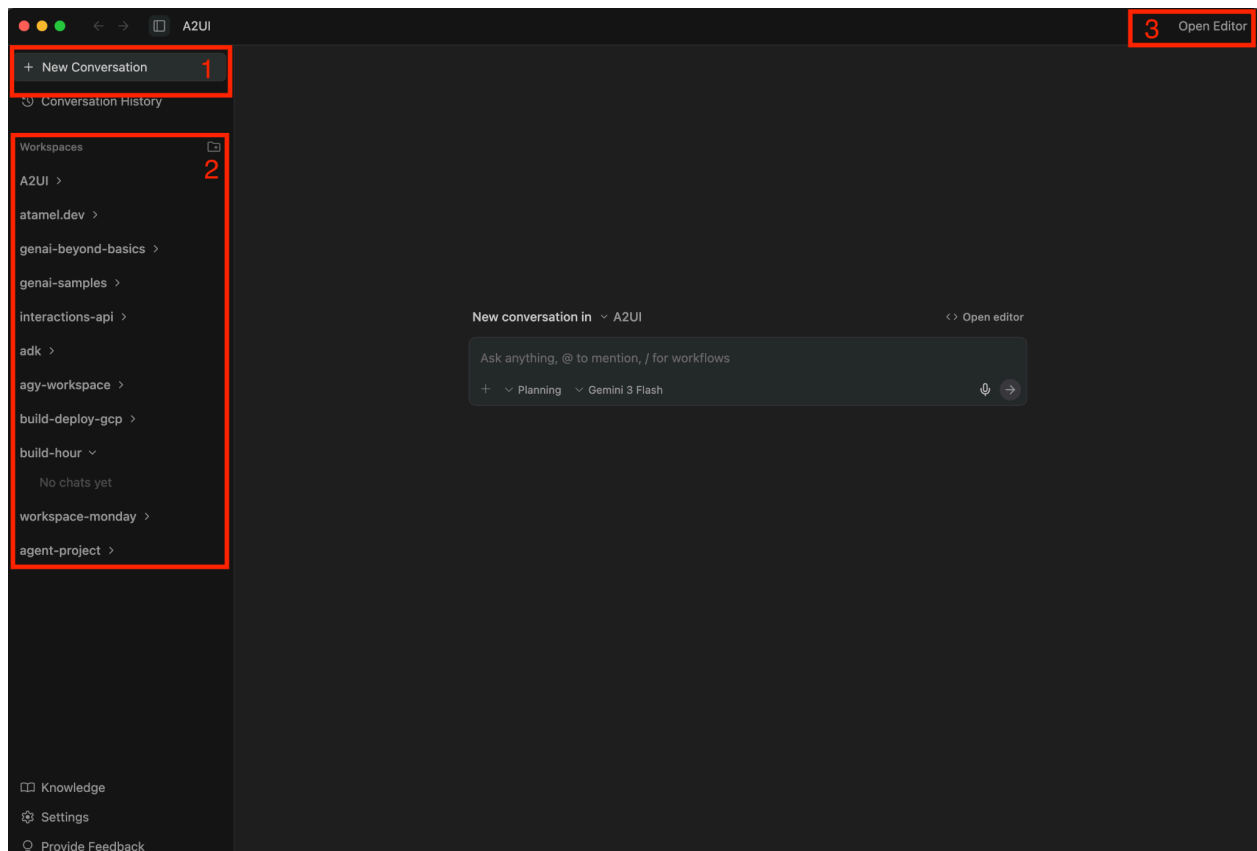


請參閱[說明文件](#)的相關說明：

- **Planning**：代理程式可以在執行工作前進行規劃。適用於深入研究、複雜工作或協作。在此模式下，代理程式會將工作歸類為工作群組、產生構件，並採取其他步驟，徹底研究、思考及規劃工作，以達到最佳品質。您會看到更多輸出內容。
- **Fast**：Agent 會直接執行工作。適用於可快速完成的簡單工作，例如重新命名變數、啟動幾個 Bash 指令，或其他較小型的本機工作。如果速度是重要因素，且工作簡單到不必擔心品質變差，這項功能就非常實用。

如果您熟悉代理程式中的思考預算和類似詞彙，可以將這項功能視為控制代理程式思考的能力，進而直接影響思考預算。我們目前會使用預設值，但請注意，在推出時，Gemini 3 Pro 模型會為每位使用者提供有限的配額，因此如果 Gemini 3 的免費配額用完，您會看到相應的訊息。

現在，讓我們花一點時間瞭解代理程式管理員 (視窗)，並掌握一些事項，清楚瞭解基本建構區塊、如何在 Antigravity 中導覽等。「Agent Manager」(代理程式管理工具) 視窗如下所示：



請參考上圖中的數字：

1. **Start Conversation**：按一下這個圖示即可發起新對話。系統會直接將你帶往顯示 **Ask anything** 的輸入欄位。
 2. **Workspaces**：我們提到工作區，以及您可以在任何工作區中工作。你隨時可以新增更多工作區，並在開始對話時選取任何工作區。
 3. **Editor View**：你隨時可以切換至編輯器檢視畫面。系統會顯示工作區資料夾和所有產生的檔案。您可以直接編輯檔案，甚至在編輯器中提供內嵌指引或指令，讓 **Agent** 根據您修改的建議/指示執行動作或進行變更。我們會在後續章節中詳細說明編輯器檢視畫面。
-

4. Antigravity 瀏覽器

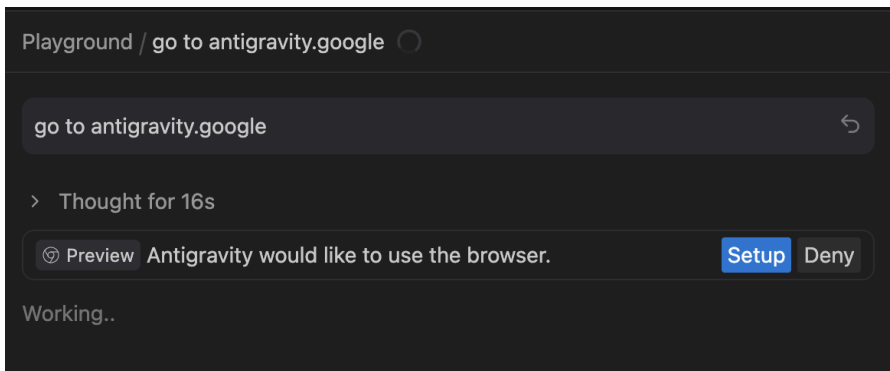
如文件所述，當代理程式想與瀏覽器互動時，會叫用瀏覽器子代理程式來處理手邊的工作。瀏覽器子代理程式會執行專門的模型，在 Antigravity 管理的瀏覽器中開啟的頁面上運作，這與您為主要代理程式選取的模型不同。

這個子代理程式有權存取各種控制瀏覽器所需的工具，包括點選、捲動、輸入、讀取控制台記錄等。Gemini 也能透過 DOM 擷取、螢幕截圖或 Markdown 剖析功能讀取開啟的網頁，以及錄製影片。

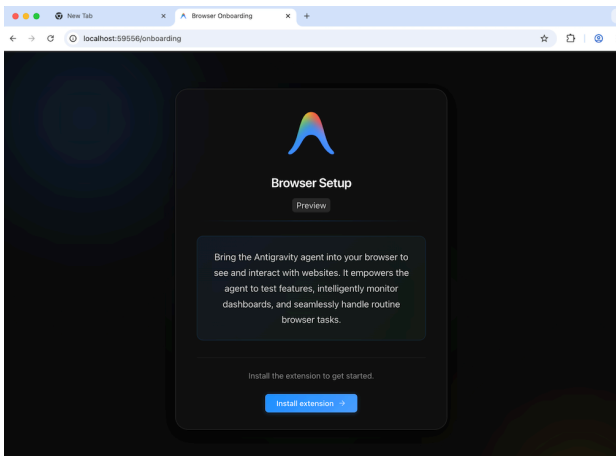
也就是說，我們需要啟動並安裝 Antigravity 瀏覽器擴充功能。我們實際發起對話並逐步操作，瞭解如何進行這項操作。

在工作區中發起新對話，並提供下列工作：`go to antigravity.google`

提交工作。你會看到代理程式分析工作，並可檢查思考過程。在某個時間點，系統會正確繼續執行，並提及需要設定瀏覽器代理程式，如下所示。按一下 `Setup`。



系統會開啟瀏覽器並顯示訊息，提示您安裝擴充功能，如下所示：



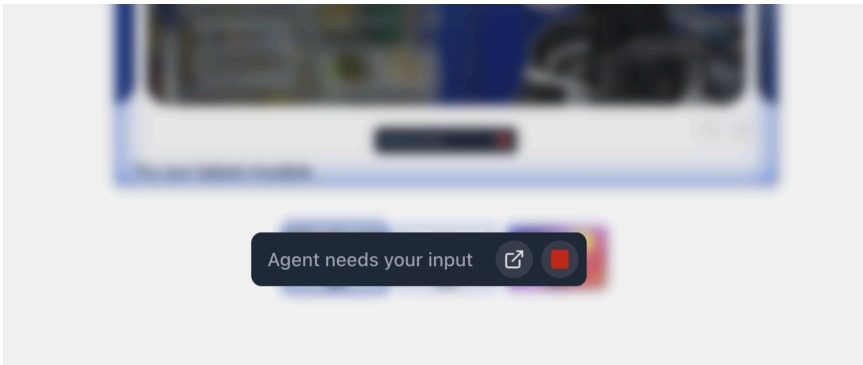
繼續操作，系統會將你導向 Chrome 擴充功能，然後即可安裝。



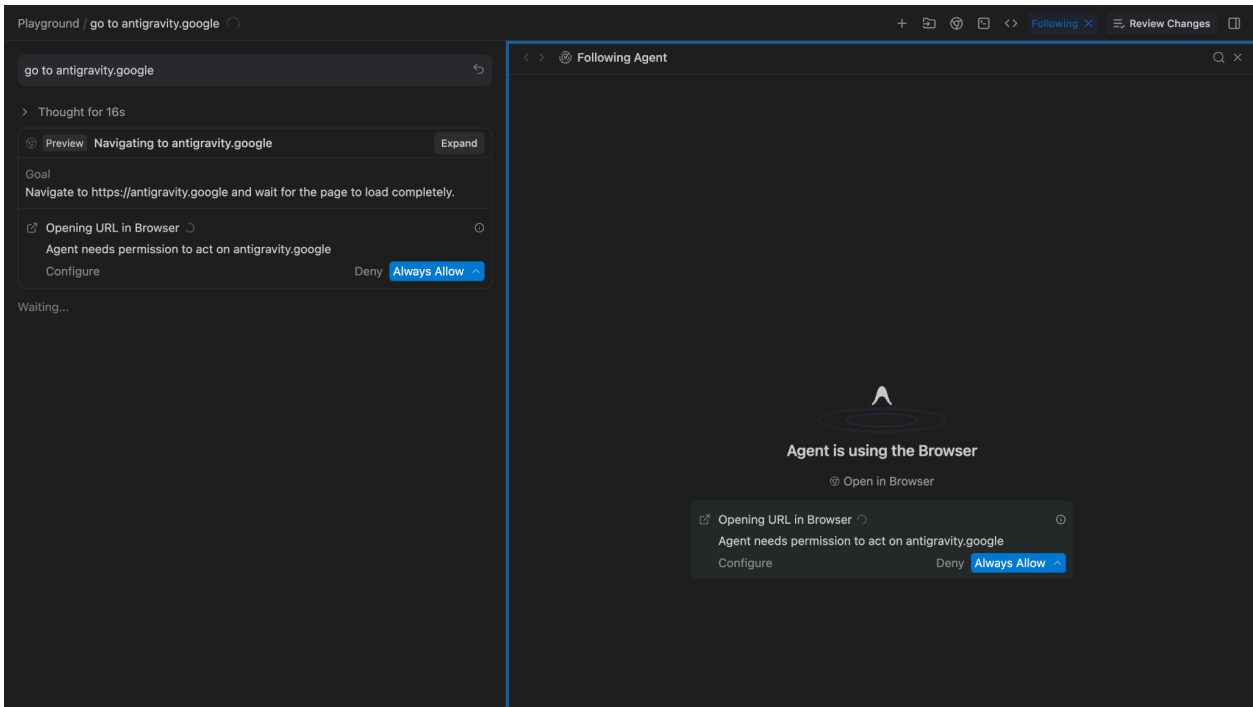
如果基於任何原因需要手動安裝擴充功能，請在 Agent Manager 中執行下列操作：

- 按一下 Antigravity 中的 Chrome 圖示 (開啟 Chrome 並使用 Antigravity 使用者設定檔)
- 在編輯器中，Chrome 圖示位於右上角
- 在「代理程式管理員」中，Chrome 圖示位於左下角
- 前往該網址，然後按一下「加到 Chrome」

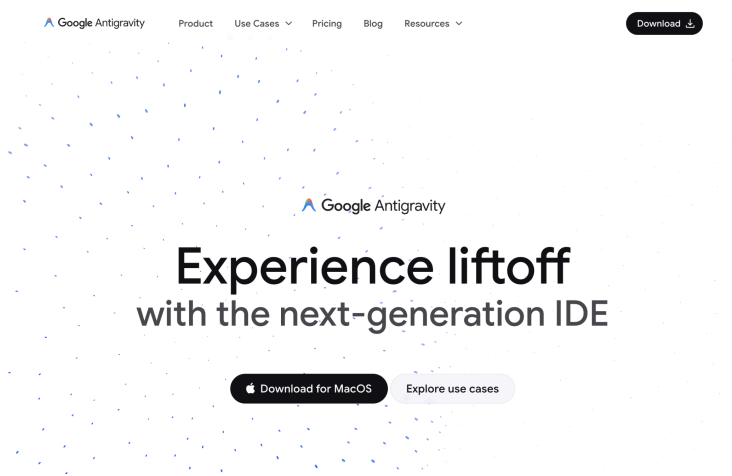
成功安裝擴充功能後，Antigravity Agent 就會開始運作，並顯示訊息，要求您授權執行工作。開啟的瀏覽器視窗中應該會顯示一些活動：



切換回 Agent Manager 檢視畫面，您應該會看到以下內容：



我們要求 Agent 前往 `antigravity.google` 網站，因此這正是我們預期的結果。授予權限後，您會發現系統已安全地導向該網站，如下所示：



5. 構件

Antigravity 會在規劃及完成工作時建立構件，藉此傳達工作內容並取得人類使用者的意見回饋。例如豐富的 Markdown 檔案、架構圖、圖片、瀏覽器錄影、程式碼差異等。

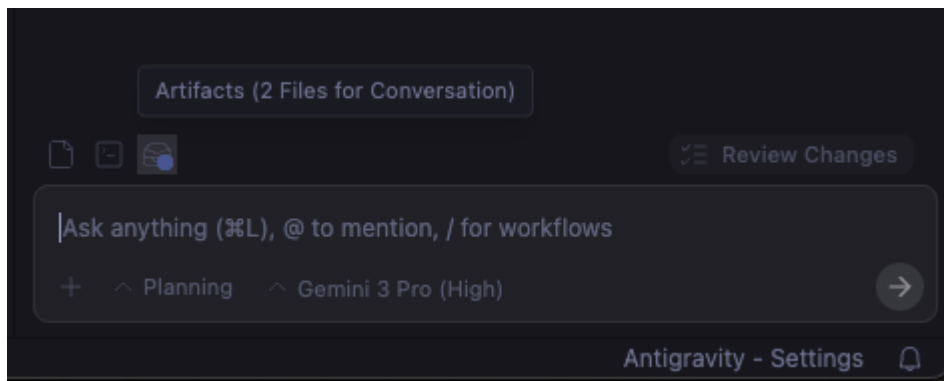
構件可解決「信任落差」問題。如果代理程式聲稱「我已修正錯誤」，開發人員先前必須閱讀程式碼才能驗證。在 Antigravity 中，代理程式會產生證明這項事實的構件。

Antigravity 主要產生的構件如下：

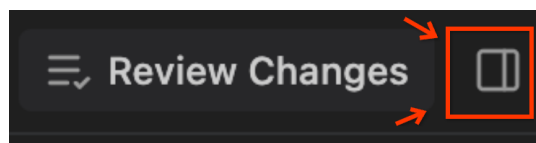
- **Task Lists**：代理程式會在編寫程式碼前生成結構化計畫。您通常不需要編輯這項計畫，但可以查看計畫，並視需要新增留言來變更計畫。
- **Implementation Plan**：用於在程式碼集內設計變更，以完成工作。這些計畫包含必要修訂內容的技術詳細資料，除非您將構件審查政策設為「一律繼續」，否則使用者必須審查這些計畫。
- **Walkthrough**：代理程式完成工作實作後，就會建立這份文件，當中會摘要說明變更內容和測試方式。
- **Code diffs**：雖然嚴格來說不是構件，但 Antigravity 也會產生程式碼差異，供您審查及加上註解。
- **Screenshots**：代理程式會擷取變更前後的 UI 狀態。
- **Browser Recordings**：如果是動態互動 (例如「點選登入按鈕、等待微調器、確認資訊主頁載入」)，專員會錄製工作階段影片。開發人員可以觀看這部影片，確認應用程式符合功能需求，不必自行執行應用程式。

系統會產生構件，並顯示在代理程式管理員和編輯器檢視畫面中。

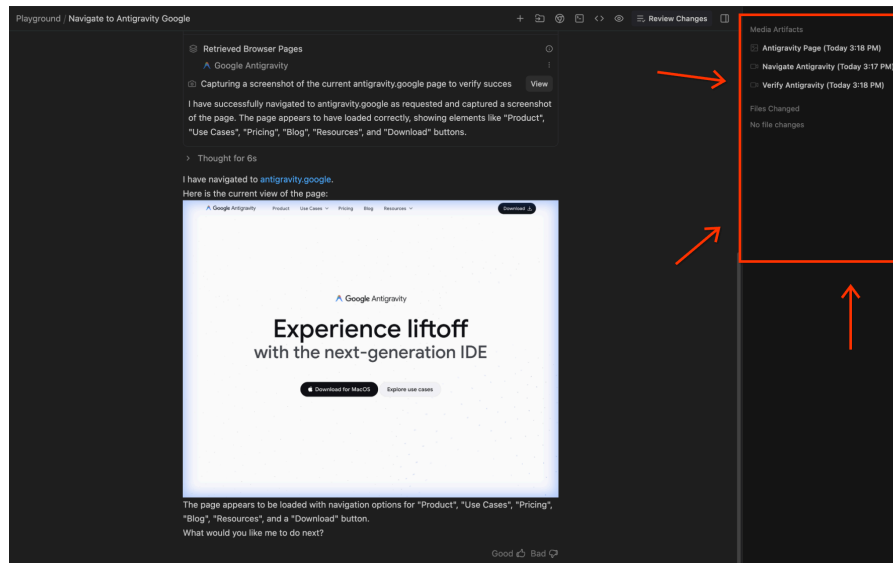
在「編輯器」檢視畫面中，按一下右下角的 **Artifacts**，即可執行下列操作：



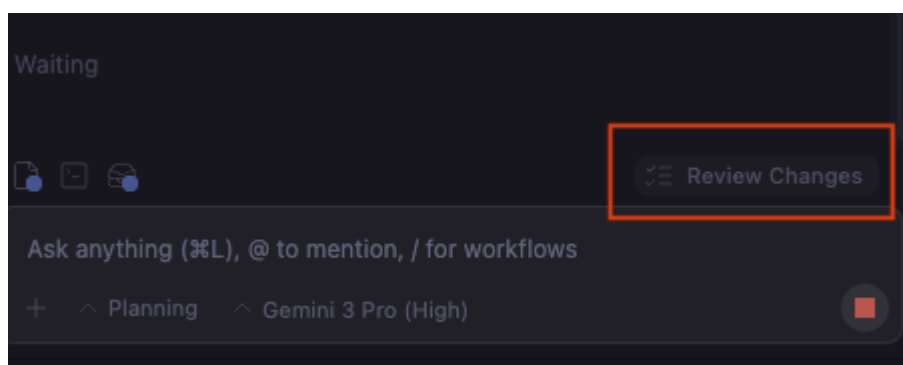
在「代理程式管理員」檢視畫面中，右上方的 **Review changes** 旁應該會顯示切換構件的按鈕，如果已開啟切換功能，則會顯示產生的構件清單：



您應該會看到如下所示的「構件」檢視畫面。在本例中，我們指示 Agent 瀏覽 `antigravity.google` 頁面，因此 Agent 擷取了該頁面的螢幕截圖，並製作了相關影片：



您可以在「編輯器」檢視畫面中查看 **Review Changes** 的程式碼差異：



開發人員可以使用「Google 文件風格的註解」，與這些構件和程式碼差異互動。你可以選取特定動作或工作，然後提供指令給代理程式。接著，代理程式會擷取這項意見回饋，並據此反覆調整。建議使用互動式 Google 文件，您可向作者提供意見回饋，作者再根據這些意見進行修改。

步驟 6 · 約 5 分鐘

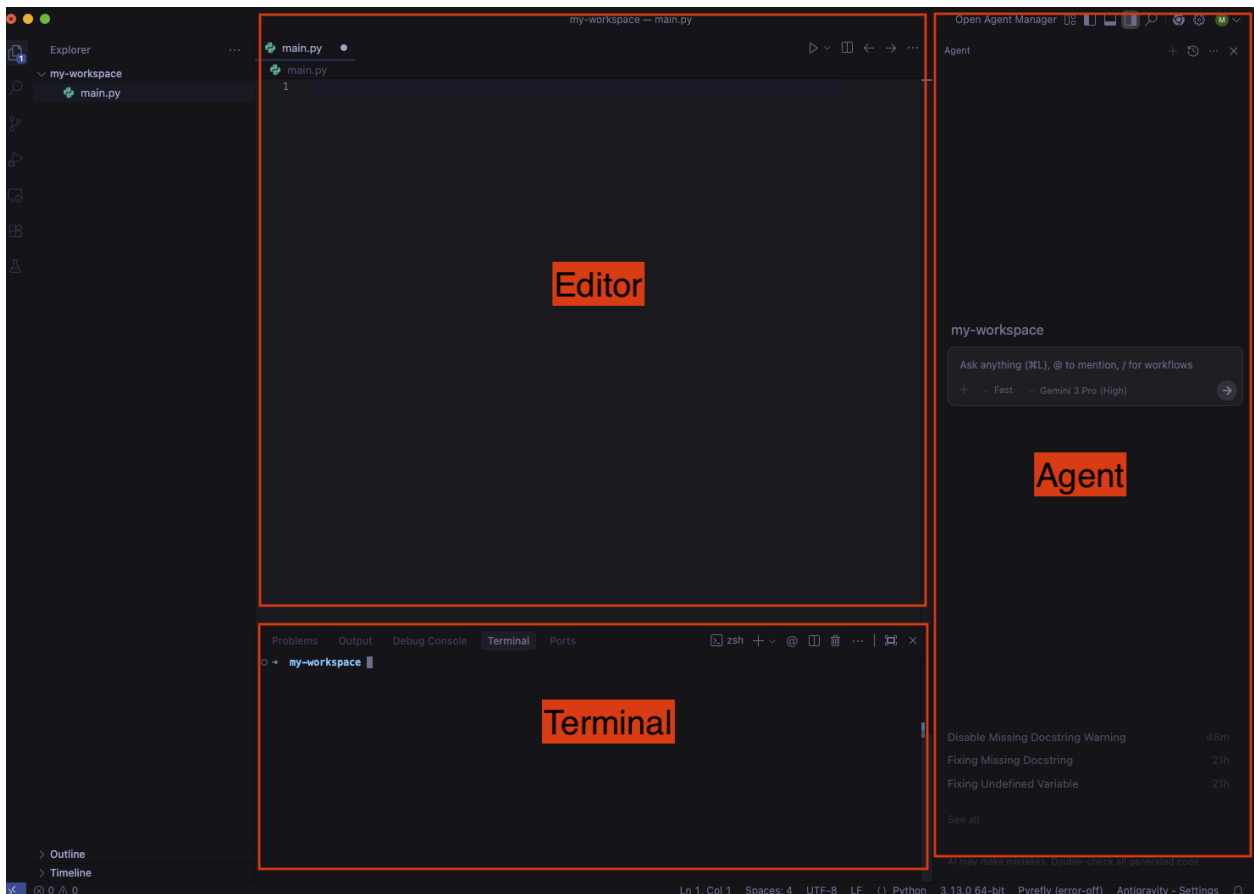
6. 編輯者

編輯器保留了 VS Code 的熟悉感，確保資深開發人員的肌肉記憶不會受到影響。包括標準檔案總管、語法醒目顯示和擴充功能生態系統。

如要前往編輯器，請按一下代理程式管理員右上方的 `Open Editor` 按鈕。

設定和擴充功能

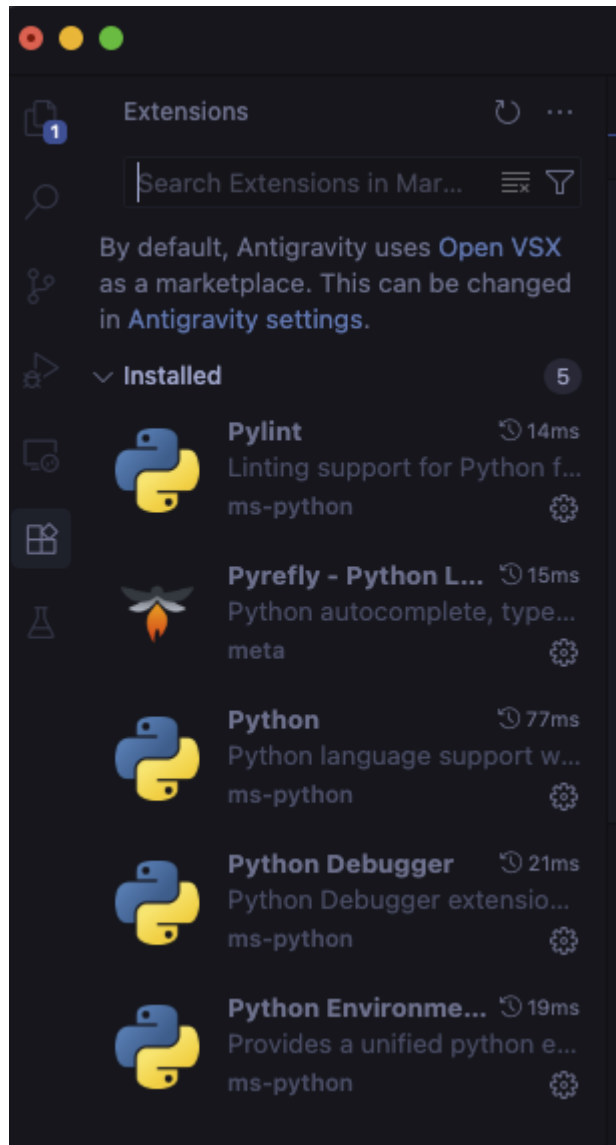
一般設定會顯示編輯器、終端機和代理程式：



如果不是這種情況，請按照下列方式切換終端機和代理程式面板：

- 如要切換終端機面板，請使用 `Ctrl + `` 快速鍵。
- 如要切換顯示/隱藏代理程式面板，請使用 `Cmd + L` 快速鍵。

此外，Antigravity 可以在設定期間安裝部分擴充功能，但視您使用的程式設計語言而定，您可能需要安裝更多擴充功能。舉例來說，如果是 Python 開發，您可能會選擇安裝下列擴充功能：



編輯者

自動完成

在編輯器中輸入程式碼時，系統會啟動智慧自動完成功能，按下 **Tab** 鍵即可接受建議：

```
main.py
main.py > ...
1 def main():
2     print("Hello, World!")
3
4 if __name__ == "__main__":
    main()
```

按 Tab 鍵即可匯入

系統會顯示匯入分頁建議，協助您新增缺少的依附元件：

```
main.py > main
1  """
2  This is a simple Python script to showcase functionality.
3  """
4
5  def main():
6      """
7      Main function.
8      """
9      print("Hello, World!")
10     print(f"My favourite number is {math.pi}")
11
12 if __name__ == "__main__":
13     main()
14
```

按 Tab 鍵跳轉

您會收到**Tab 鍵跳轉**建議，將游標移至程式碼中的下一個邏輯位置：

```
import math

def main():
    """
    Main function.
    """
    print("Hello, World!")
    print(f"My favourite number is {math.pi}")

if __name__ == "__main__":
    main()
```

指令

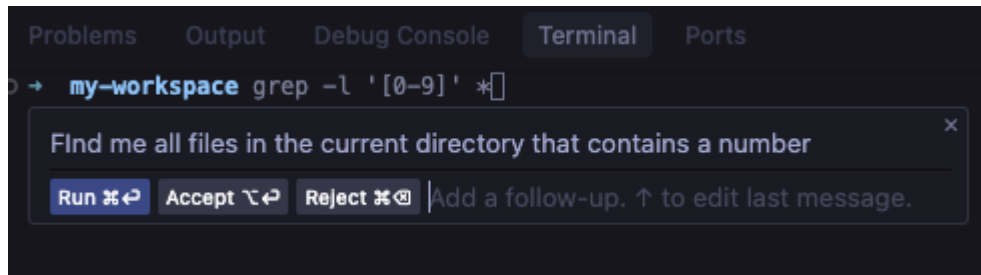
您可以在編輯器或終端機中，使用 **Cmd + I** 觸發指令，以自然語言取得內嵌完成建議。

在編輯器中，您可以要求計算費波那契數的方法，然後接受或拒絕：

```
Write me a method to calculate fibonacci numbers
Accept ⌘↵ Reject ⌘⌫ Add a follow-up. ↑ to edit last message.

def fibonacci(n):
    """
    Calculates the nth Fibonacci number.
    """
    if n <= 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)
```

在終端機中，你可以取得終端機指令建議：



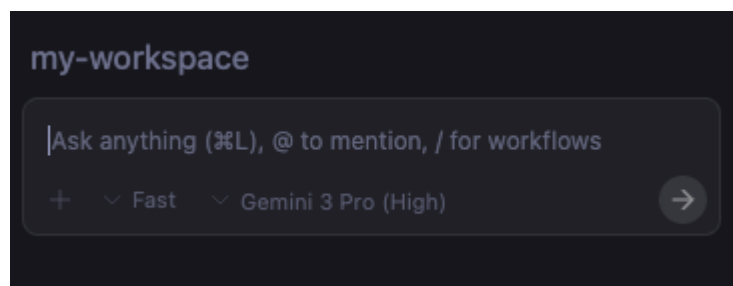
代理程式側邊面板

在編輯器中，你可以透過多種方式切換代理側邊面板。

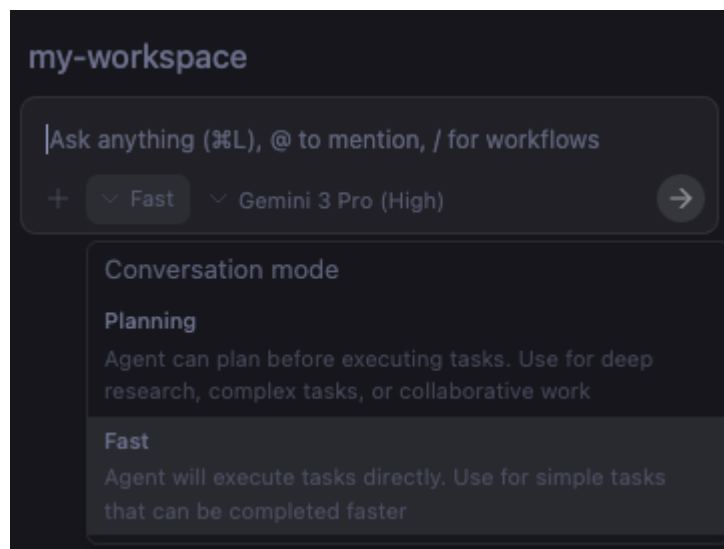
手動開啟

您可以使用 `Cmd + L` 快速鍵，手動切換右側的代理程式面板。

你可以開始提問、使用 `@` 納入更多脈絡 (例如檔案、目錄、MCP 伺服器)，或使用 `/` 參照工作流程 (已儲存的提示)：

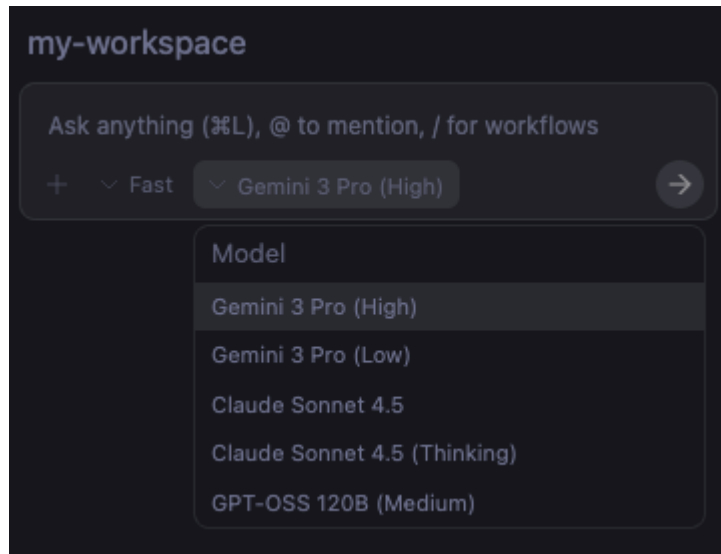


你也可以選擇兩種對話模式：`Fast` 或 `Planning`：



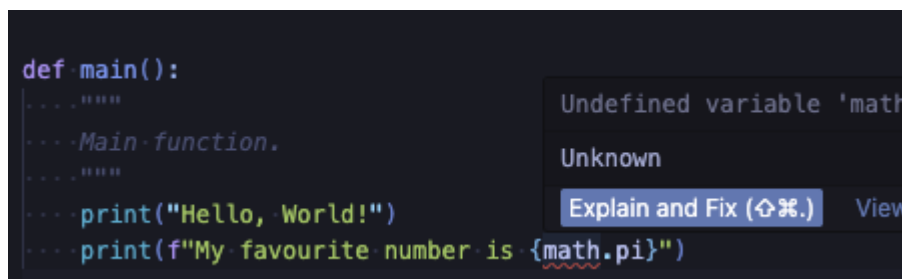
`Fast` 適合快速完成工作，`Planning` 則適合處理較複雜的工作，代理程式會建立計畫供您核准。

你也可以選擇其他對話模型：



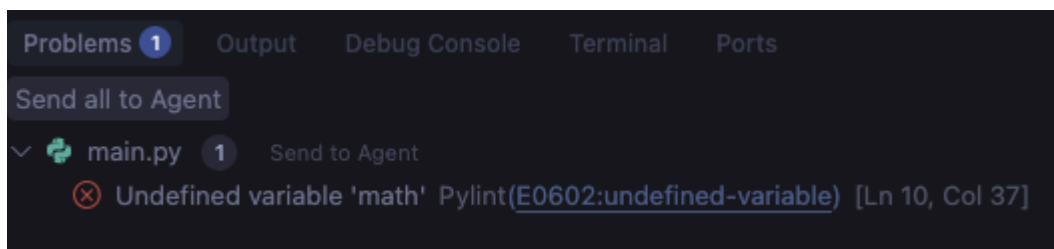
說明並修正

如要觸發代理，也可以將游標懸停在問題上，然後選取 `Explain and fix`：



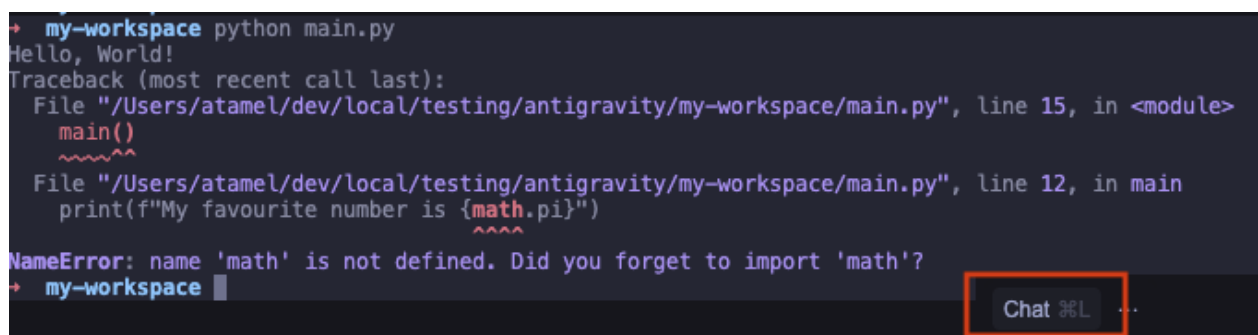
將問題傳送給代理

你也可以前往「Problems」部分，然後選取「Send all to Agent」，讓服務專員嘗試修正這些問題：



將終端機輸出內容傳送給代理

你甚至可以選取終端機輸出內容的一部分，然後按 `Cmd + L` 將其傳送給代理程式：



在編輯器和代理程式管理員之間切換

在編輯器模式中，您可以隨時按一下右上方的 `Open Agent Manager` 按鈕，切換至完整代理程式管理員模式；在代理程式管理員模式中，則可按一下右上方的 `Open Editor` 按鈕，切換回編輯器模式。

你也可以使用 `Cmd + E` 鍵盤快速鍵，在兩種模式之間切換。

7. 提供意見

Antigravity 的核心功能是在體驗的每個階段輕鬆收集意見回饋。代理執行工作時，會建立不同的構件：

- 導入計畫和工作清單 (編碼前)
- 程式碼差異 (生成程式碼時)
- 逐步驗證結果 (編碼後)

Antigravity 會透過這些構件傳達計畫和進度。更重要的是，您還能以 Google 文件註解的形式，向服務專員提供意見回饋。這項功能非常實用，可有效引導代理程式朝您期望的方向發展。

讓我們試著建立簡單的待辦事項應用程式，並瞭解如何在此過程中向 Antigravity 提供意見。

規劃模式

首先，請確認 Antigravity 處於 `Planning` 模式 (而非 `Fast` 模式)。您可以在代理程式側邊面板的對話中選取此模式。這樣一來，Antigravity 就會在開始編寫程式碼前，先建立導入計畫和工作清單。然後嘗試輸入提示，例如 `Create a todo list web app using Python`。這項操作會啟動代理程式，開始規劃及製作實作計畫。

實作計畫

實作計畫會概略說明 Antigravity 的意圖、使用的技術堆疊，以及建議變更的概要說明。

```
Implementation Plan - Python Todo App
Goal
Create a simple, functional, and aesthetically pleasing Todo List web application using Python (Flask).

Tech Stack
Backend: Python with Flask
Frontend: HTML5, CSS3 (Vanilla), Jinja2 templates
...
```

你也可以在這裡提供意見回饋。在本範例中，代理程式想使用 Flask 做為 Python 網頁架構。我們可以在實作計畫中新增註解，改用 FastAPI。新增註解後，請提交註解，或要求 Antigravity `Proceed` 提供更新後的導入計畫。

工作清單

更新實施計畫後，Antigravity 會建立工作清單。這是 Antigravity 建立及驗證應用程式時會採取的具體步驟清單。

```
Task Plan
Create requirements.txt
Create directory structure (static/css, templates)
Create static/css/style.css
Create templates/index.html
Create main.py with FastAPI setup and Database logic
Verify application
```

這是第二個提供意見回饋的位置。

舉例來說，在我們的用途中，您可以新增下列註解，加入更詳細的驗證說明：`Verify application by adding, editing, and deleting a todo item and taking a screenshot.`

提供意見回饋時，請參考下列提示：

1. 在計畫或工作新增註解時，請務必記得提交註解。這會觸發 Antigravity 更新方案。
2. 有時 Antigravity 會在建立實作計畫和工作清單後，直接開始編碼，不會等待您確認。在這些情況下，您仍可對計畫/工作加上註解，然後再次提交。我們會在之後更新程式碼和逐步操作說明。您也可以停止編碼工作、在計畫/工作新增註解，然後再試一次。

程式碼變更

此時，Antigravity 會在新檔案中產生一些程式碼。您可以在代理程式對話側邊面板中 `Accept all` 或 `Reject all` 這些變更，不必查看詳細資料。

您也可以點選 `Review changes` 查看變更詳細資料，並在程式碼中新增詳細註解。舉例來說，我們可以在 `main.py` 中新增下列註解：`Add basic comments to all methods`

這是使用 Antigravity 疊代程式碼的絕佳方式。

逐步操作說明

Antigravity 完成編碼後，會啟動伺服器並開啟瀏覽器來驗證應用程式。這時會進行一些手動測試，例如新增工作、更新工作等。這一切都要歸功於 Antigravity 瀏覽器擴充功能。最後，這項工具會建立導覽檔案，總結驗證應用程式的作業，包括螢幕截圖或含有瀏覽器記錄的驗證流程。

您也可以在導覽中對螢幕截圖或瀏覽器錄影內容加上註解。例如，我們可以新增註解 `Change the blue theme to orange theme` 並提交。提交註解後，Antigravity 會進行變更、驗證結果，並更新逐步解說

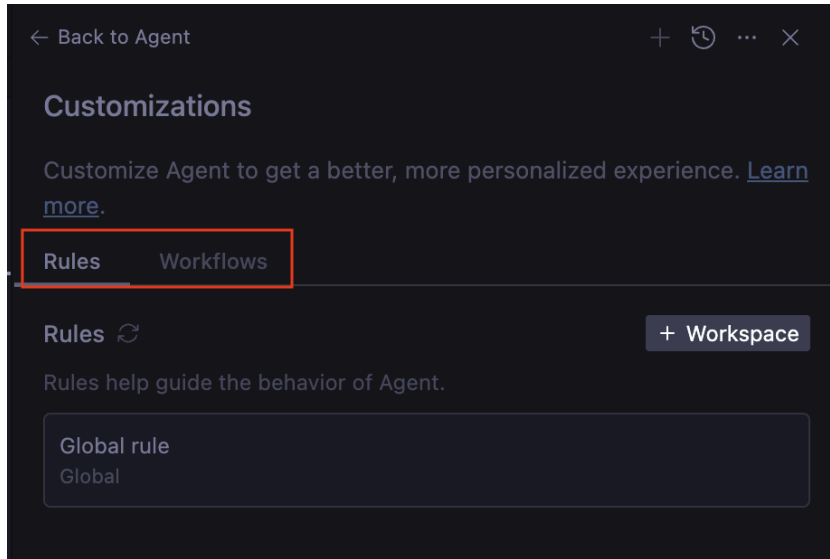
復原變更

最後，如果對變更不滿意，可以在每個步驟後從對話中復原。只要在對話中選擇  `Undo changes up to this point` 即可。

8. 規則和工作流程

Antigravity 提供幾種自訂選項：「規則」和「工作流程」。

在編輯器模式中，按一下右上角的 `...`，然後選擇 `Customizations`，您會看到 `Rules` 和 `Workflows`：



規則可引導代理的行為。您可以提供這些指引，確保代理程式在生成程式碼和測試時遵循指引。舉例來說，您可能希望代理程式遵循特定程式碼樣式，或一律記錄方法。您可以將這些資訊新增為規則，代理程式就會將其納入考量。

工作流程是儲存的提示，您可以在與代理程式互動時，透過 `/` 視需要觸發。這些工具也會引導代理程式的行為，但會由使用者視需要觸發。

舉例來說，**規則**比較像是系統指令，而**工作流程**則比較像是可隨選的已儲存提示。

「規則」和「工作流程」都可以套用至全域或個別工作區，並儲存至下列位置：

- 全域規則：`~/.gemini/GEMINI.md`
- 全域工作流程：`~/.gemini/antigravity/global_workflows/<YOUR_WORKFLOW_NAME>.md`
- 工作區規則：`your-workspace/.agents/rules/`
- 工作區工作流程：`your-workspace/.agents/workflows/`

我們在工作區中新增一些規則和工作流程。

新增規則

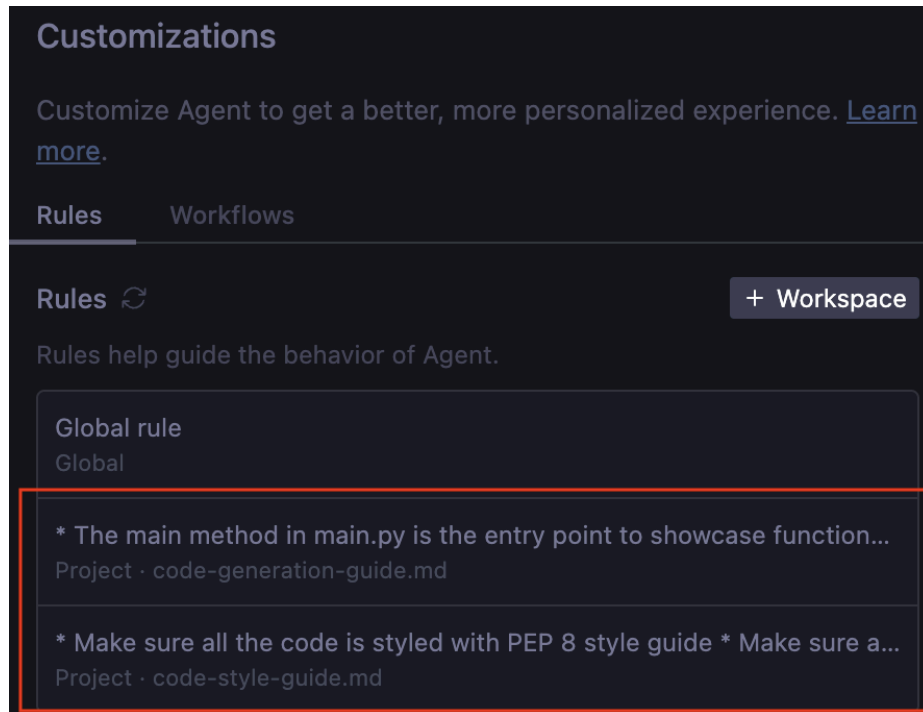
首先，新增程式碼樣式規則。前往 `Rules`，然後選取 `+Workspace` 按鈕。使用下列程式碼樣式規則為其命名，例如 `code-style-guide`：

```
* Make sure all the code is styled with PEP 8 style guide
* Make sure all the code is properly commented
```

接著，我們新增另一項規則，確保程式碼以模組化方式生成，並在 `code-generation-guide` 規則中提供範例：

- * The main method in main.py is the entry point to showcase functionality.
- * Do not generate code in the main method. Instead generate distinct functionality in a new file (eg. feature_x.py)
- * Then, generate example code to show the new functionality in a new method in main.py (eg. example_feature_x) and simply call that method from the main method.

這兩項規則會儲存並準備就緒：



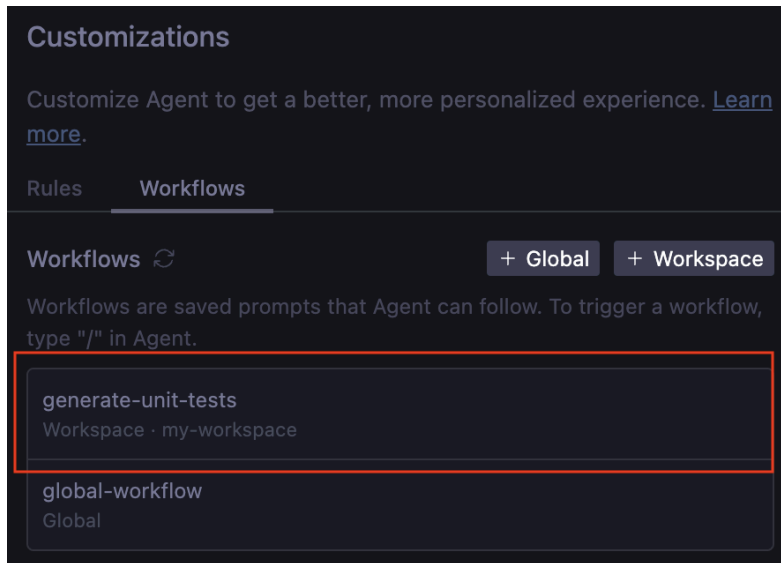
新增工作流程

我們也來定義生成單元測試的工作流程。這樣一來，我們就能在對程式碼感到滿意時觸發單元測試 (而不是讓代理程式一直生成單元測試)。

前往 **Workflows**，然後選取 **+Workspace** 按鈕。使用下列內容為其命名，例如 **generate-unit-tests**：

- * Generate unit tests for each file and each method
- * Make sure the unit tests are named similar to files but with test_ prefix

工作流程也已準備就緒：



立即試用

現在來看看規則和工作流程的實際運作情形。在工作區中建立架構 `main.py` 檔案：

```
def main():  
    pass  
  
if __name__ == "__main__":  
    main()
```

現在請前往代理程式對話視窗，詢問代理程式：`Implement binary search and bubble sort.`

一兩分鐘後，您應該會在工作區中看到三個檔案：`main.py`、`bubble_sort.py`、`binary_search.py`。您也會發現所有規則都已實作：主要檔案不會雜亂無章，且包含範例程式碼；每個功能都在各自的檔案中實作；所有程式碼都已加上註解，且風格良好：

```

from binary_search import binary_search, binary_search_recursive
from bubble_sort import bubble_sort, bubble_sort_descending

def example_binary_search():
    """
    Demonstrate binary search algorithm with various test cases.
    """
    ...

def example_bubble_sort():
    """
    Demonstrate bubble sort algorithm with various test cases.
    """
    ...

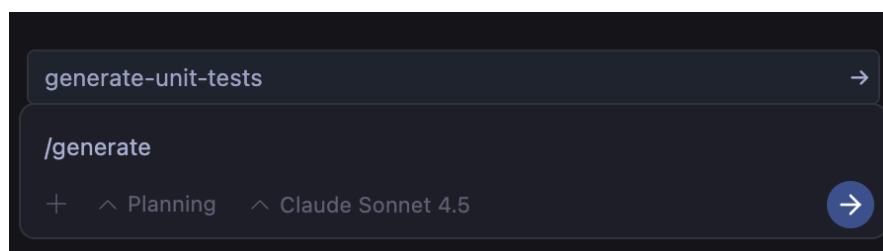
def main():
    """
    Main entry point to showcase functionality.
    """
    example_binary_search()
    example_bubble_sort()
    print("\n" + "=" * 60)

if __name__ == "__main__":
    main()

```

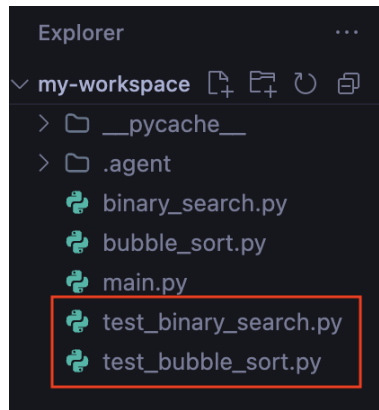
我們對程式碼感到滿意，現在來看看是否可以觸發產生單元測試工作流程。

前往對話並開始輸入 `/generate`，Antigravity 就會瞭解我們的工作流程：



選取 `generate-unit-tests` 並輸入。幾秒後，工作區就會出現新檔

案：`test_binary_search.py`、`test_bubble_sort.py`，其中已實作多項測試！



太棒了！

9. 技能

雖然 Antigravity 的基礎模型 (例如 Gemini) 是功能強大的通才，但並不瞭解您的特定專案背景或團隊標準。將所有規則或工具載入代理程式的內容視窗，會導致「工具膨脹」、成本增加、延遲和混淆。

Antigravity Skills 會透過漸進式揭露解決這個問題。**技能**是專業知識的特殊套件，在需要時才會啟動。只有在特定要求符合技能說明時，才會載入代理程式的情境。

結構和範圍

技能是以目錄為基礎的套件。您可以視需求在兩個範圍中定義這些變數：

- 全域範圍 (`~/gemini/antigravity/skills/`)：適用於所有專案 (例如「Format JSON」、「General Code Review」)。
- 工作區範圍 (`<workspace-root>/agents/skills/`)：僅適用於特定專案 (例如「Deploy to this app's staging」(部署至這個應用程式的暫存環境)、「Generate boilerplate for this specific framework」(為這個特定架構產生樣板))。

技能剖析

典型的技能目錄如下所示：

```
my-skill/
├── SKILL.md      # (Required) metadata & instructions.
├── scripts/      # (Optional) Python or Bash scripts for execution.
├── references/   # (Optional) text, documentation, or templates.
└── assets/       # (Optional) Images or logos.
```

現在新增一些技能。

程式碼審查技能

這項技能僅提供指令，也就是說，我們只需要建立 `SKILL.md` 檔案，其中會包含中繼資料和技能指令。讓我們建立**全域技能**，提供詳細資料給代理程式，以審查程式碼變更是否有錯誤、樣式問題和最佳做法。

首先，請建立目錄，用來存放這個全域技能。

```
mkdir -p ~/gemini/antigravity/skills/code-review
```

在上述目錄中建立 `SKILL.md` 檔案，並加入以下內容：

```
---
name: code-review
description: Reviews code changes for bugs, style issues, and best practices. Use when
reviewing PRs or checking code quality.
---

# Code Review Skill

When reviewing code, follow these steps:

## Review checklist

1. Correctness: Does the code do what it's supposed to?
2. Edge cases: Are error conditions handled?
3. Style: Does it follow project conventions?
4. Performance: Are there obvious inefficiencies?

## How to provide feedback

- Be specific about what needs to change
- Explain why, not just what
- Suggest alternatives when possible
```

請注意，上方的 `SKILL.md` 檔案頂端包含中繼資料 (名稱和說明)，接著是指令。代理程式載入時，只會讀取您設定的技能中繼資料，並在必要時載入技能的指令。

立即試用

建立名為 `demo_bad_code.py` 的檔案，並在當中加入下列內容：

```

import time

def get_user_data(users, id):
    # Find user by ID
    for u in users:
        if u['id'] == id:
            return u
    return None

def process_payments(items):
    total = 0
    for i in items:
        # Calculate tax
        tax = i['price'] * 0.1
        total = total + i['price'] + tax
        time.sleep(0.1) # Simulate slow network call

    return total

def run_batch():
    users = [{'id': 1, 'name': 'Alice'}, {'id': 2, 'name': 'Bob'}]
    items = [{'price': 10}, {'price': 20}, {'price': 100}]

    u = get_user_data(users, 3)
    print("User found: " + u['name']) # Will crash if None

    print("Total: " + str(process_payments(items)))

if __name__ == "__main__":
    run_batch()

```

詢問虛擬服務專員：`review the @demo_bad_code.py file`。代理程式應找出 `code-review` 技能、載入詳細資料，然後根據 `code-review/SKILL.md` 檔案中的指示執行動作。

輸出範例如下所示：

Following the **Code Review Skill** guidelines, here is the review for `demo_bad_code.py` :

1. Correctness

Issue: Potential runtime crash in `run_batch` .

```
python
24:     u = get_user_data(users, 3)
25:     print("User found: " + u['name']) # Will crash if None
```

Why: When `get_user_data` returns `None` (which happens here since ID 3 doesn't exist), accessing `u['name']` will raise a `TypeError: 'NoneType' object is not subscriptable`. **Suggestion:** Check if `u` is not `None` before accessing its keys.

```
python
if u:
    print(f"User found: {u['name']}")
else:
    print("User not found")
```

2. Edge Cases

Issue: `get_user_data` linear search. **Why:** While fine for small lists, `get_user_data` iterates through the whole list. If used frequently or with large datasets, a dictionary lookup would be safer and more performant. **Suggestion:** Consider indexing users by ID if this lookup happens often.

3. Style

Issue: Shadowing built-in `id` .

```
python
3: def get_user_data(users, id):
```

Why: `id` is a built-in Python function. Shadowing it can lead to confusing bugs if you ever need to use the original function. **Suggestion:** Rename the argument to `user_id` .

Issue: String concatenation.

```
python
25:     print("User found: " + u['name'])
```

Why: String concatenation using `+` is less readable and slightly less efficient than f-strings. **Suggestion:** Use f-strings: `print(f"User found: {u['name']}")` .

4. Performance

Issue: Blocking network call simulation in loop.

```
python
16:         time.sleep(0.1) # Simulate slow network call
```

Why: Executing a 0.1s wait for every item sequentially will essentially block the thread. If `items` has 100 entries, this function takes 10 seconds. **Suggestion:** In a real scenario, network calls should be batched (send all items at once) or performed asynchronously to avoid blocking the main execution flow.

程式碼標頭範本技能

有時技能需要使用大段靜態文字 (例如授權標題)。直接將這段文字放入提示詞中會造成浪費。我們將其放在 `resources/` 資料夾中，並指示代理程式只在必要時讀取。

首先，請建立目錄，用來存放這個工作區技能。

```
mkdir -p .agents/skills/license-header-adder/resources
```

使用授權文字建立 `.agents/skills/license-header-adder/resources/HEADER.txt` :

```
/*
 * Copyright (c) 2026 YOUR_COMPANY_NAME LLC.
 * All rights reserved.
 * This code is proprietary and confidential.
 */
```

建立 `.agents/skills/license-header-adder/SKILL.md` 檔案，並加入下列內容：


```
---
name: license-header-adder
description: Adds the standard corporate license header to new source files.
---

# License Header Adder

This skill ensures that all new source files have the correct copyright header.

## Instructions
1. Read the Template: Read the content of `resources/HEADER.txt`.
2. Apply to File: When creating a new file, prepend this exact content.
3. Adapt Syntax:
    - For C-style languages (Java, TS), keep the `/* */` block.
    - For Python/Shell, convert to `#` comments.
```

立即試用

向服務專員提出下列問題：`Create a new Python script named data_processor.py that prints 'Hello World'.`

代理程式會讀取範本、將 C 樣式的註解轉換為 Python 樣式，並自動將註解加到新檔案開頭。

建立這些技能後，您就能將通用的 Gemini 模型轉變為專案專家。您已將最佳做法編碼，無論是遵循程式碼審查指南或授權標頭，現在，AI 代理程式會自動瞭解團隊的工作方式，不必再重複提示 AI「記得新增授權」或「修正提交格式」。

10. 安全性和控管機制

授予 AI 代理程式檔案系統、終端機和瀏覽器存取權是一把雙面刃。這項功能可自主生成、偵錯及部署程式碼，但也會開啟提示詞注入和資料竊取的向量。Antigravity 提供各種安全性控制設定，可解決這個問題。

前往 Antigravity 設定，查看「代理程式」下方的設定。

構件政策

第一項是構件審查政策。指定代理程式在要求審查構件時的行為。

政策模式	說明
一律繼續	Agent 絕不會要求審查。這可讓代理程式發揮最大自主性，但代理程式也最有可能在不安全或遭注入的構件內容上運作
代理程式決定	代理會根據工作複雜度和使用者偏好，決定何時要求審查
請客戶提供評論	Agent 一律要求審查

建議將這項政策設為 `Asks for Review`。

終端政策和沙箱

下一個是終端機指令相關政策。

有 `Terminal Command Auto Execution` 政策。這項設定會決定代理程式在執行 Shell 指令時的自主性：

政策模式	說明
提出審查申請	服務專員一律會先要求確認，再執行終端機指令 (允許清單中的指令除外)
一律繼續	服務專員執行終端機指令前，一律不會要求確認 (拒絕清單中的指令除外)。這可讓代理程式在長時間內運作，無須介入，但代理程式執行不安全終端機指令的風險也最高。

建議您一開始選擇 `Request Review`，然後逐步允許代理在每個工作區中執行更多終端機指令。

此外，您也應開啟 `Enable Terminal Sandbox` 專區。啟用後，終端機指令會受到沙箱限制。

檔案存取政策

根據預設，代理程式只能存取工作區中的檔案。這是不錯的做法，可限制代理程式可存取的檔案。如需更多資訊，請開啟 `Agent Non-Workspace File Access` 切換鈕。啟用後，代理程式就能自動查看及編輯目前工作區以外的檔案。

注意：這項設定可讓代理程式存取其他可能相關的資訊，但也會允許代理程式存取憑證檔案、密碼和其他工作區外的檔案，而這些檔案可能會成為惡意行為者發動提示詞注入攻擊或其他漏洞攻擊的目標。

代理程式權限

Antigravity 採用統一的權限系統，控管 Agent 能代您執行的動作。每個動作都以權限資源表示，可放入下列其中一個清單：

- **允許：**系統會自動核准動作，不會顯示提示。
- **拒絕：**立即封鎖動作。
- **詢問：**代理程式會暫停，並要求您核准後再繼續。

「允許」、「拒絕」或「詢問」清單中的每個項目都採用以下格式：`action(target)`

其中 `action` 是支援的動作類型之一，而 `target` 是說明權限涵蓋範圍的模式。

動作	目標格式	比對
<code>command</code>	<code>command(prefix) or command(*)</code>	依前置字串比對指令。 <code>command(git)</code> 會比對 <code>git add</code> 、 <code>git commit</code> 等。
<code>read_file</code>	<code>read_file(/path)</code>	比對檔案或目錄下的所有項目。路徑必須是常值和絕對路徑。不支援 Glob (<code>*.go</code>)、規則運算式和 <code>~</code> 。
<code>write_file</code>	<code>write_file(/path)</code>	與 <code>read_file</code> 相同。也隱含涵蓋相同路徑的 <code>read_file</code> 。
<code>read_url</code>	<code>read_url(domain) or read_url(*)</code>	比對網域和所有子網域。不符合網址路徑。
<code>mcp</code>	<code>mcp(server/tool), mcp(server/), or mcp()</code>	依據確切的伺服器名稱比對。 <code>server/*</code> 涵蓋該伺服器上的所有工具。

詳情請參閱[說明文件](#)。

測試許可清單

「允許清單」主要搭配「要求審查」政策使用。這代表正向安全模式，也就是除非明確允許，否則一律禁止。這是最安全的設定。

前往 `Permissions`，然後前往 `Always Allow` 區段並新增下列內容：`command(ls)`

然後詢問代理並觀察其行為：

- 詢問虛擬服務專員：`List the files in this directory`。
- 代理程式會 `ls` 自動執行。

- 詢問虛擬服務專員：Delete the <some file> 。
- 服務專員會嘗試 `rm <filepath>`，但 Antigravity 會強制進行使用者審查，因為 `rm` 不在許可清單中。Antigravity 應該會在執行指令前要求您提供權限。

注意：您可能需要重新啟動 Antigravity，許可清單才會生效。

測試拒絕清單

拒絕清單是「一律繼續」政策的保障措施。這代表負面安全模型，也就是允許所有項目，除非明確禁止。這項做法需要開發人員預測所有可能的危險，雖然風險較高，但速度最快。

前往 `Permissions`，然後前往 `Always Deny` 區段並新增下列內容：

- `command(rm)`

然後詢問代理並觀察其行為：

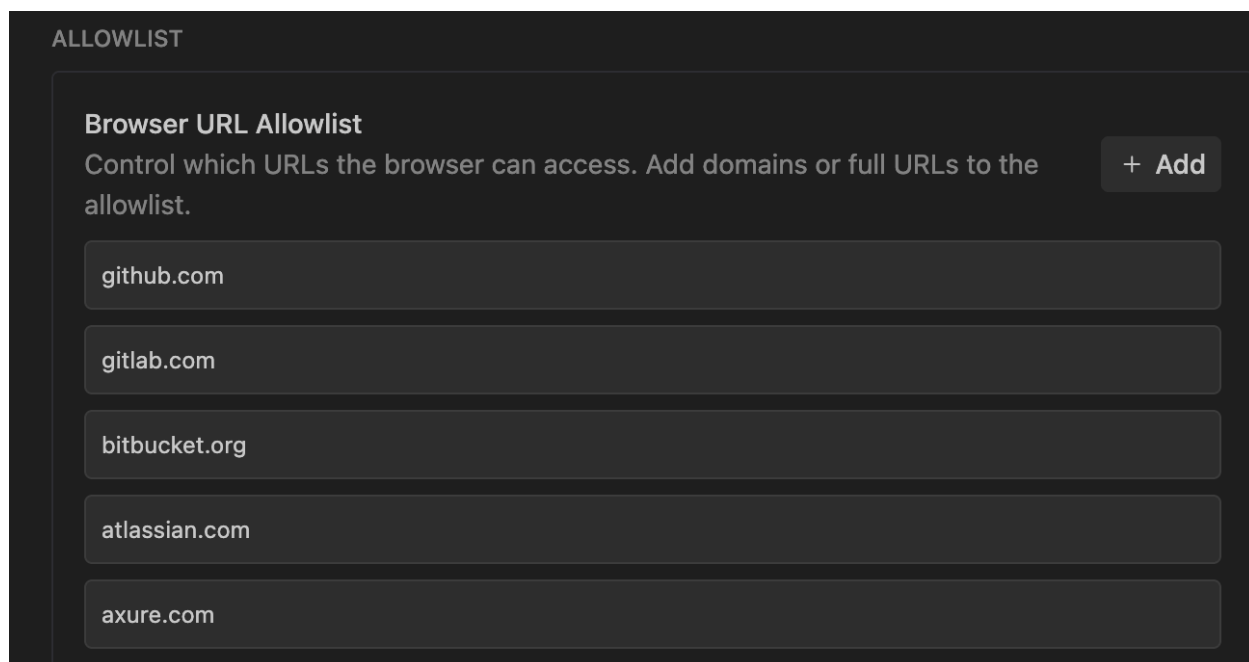
- 詢問虛擬服務專員：Create a `main.py` 。
- 代理程式會建立檔案，不會發生任何問題。
- 詢問虛擬服務專員：Delete the `main.py` file 。
- 執行反重力區塊，並提示您手動核准。

注意：您可能需要重新啟動 Antigravity，拒絕清單才會生效。

瀏覽器政策

Antigravity 的網路瀏覽能力是超能力，但也是安全漏洞。如果專員造訪遭入侵的文件網站，可能會遇到提示詞注入攻擊。為防範這種情況，您可以為瀏覽器代理程式實作瀏覽器網址許可清單。

如要查看目前的設定，請前往 `Antigravity - Settings`，然後點選 `Browser`。您應該會看到 `Browser URL Allowlist` 部分，可在此新增其他網址：



11. 結論

恭喜！您已成功安裝 Antigravity、設定環境，並瞭解如何控制代理程式。

後續步驟 如要瞭解 Antigravity 如何建構實際應用程式，請參閱下列程式碼研究室：

- 使用 [Google Antigravity 建構](#)：本程式碼實驗室說明如何建構多個應用程式，包括動態會議網站和效率提升應用程式。
- 使用 [Antigravity 建構及部署至 Google Cloud](#)：本程式碼實驗室說明如何設計、建構無伺服器應用程式，並部署至 Google Cloud。

參考文件

- 官方網站：<https://antigravity.google/>
 - 說明文件：<https://antigravity.google/docs>
 - 用途：<https://antigravity.google/use-cases>
 - 下載：<https://antigravity.google/download>
 - Google Antigravity YouTube 頻道：<https://www.youtube.com/@googleantigravity>
-